

Практична робота з Python

Тема: функції — параметри, значення за замовчуванням, docstring та анотації типів

Виконав: Krilovets Nikita, група IT-31

Персональні дані: name = Nikita, surname = Krilovets, group = IT-31, y = 2009, c = 9, n = 6.

Завдання 1. Функції з параметрами та без

Створено функцію print_card() без параметрів, яка виводить дані зі глобальних змінних. Створено функцію print_card_args() з параметрами name, surname, group, year, яку викликано кілька разів. Продemonстровано виклик функції за допомогою позиційних аргументів та іменованих (keyword) аргументів, а також повторний виклик з тими самими значеннями групи.

```
task1.py
1  name = "Nikita"
2  surname = "Krilovets"
3  group = "IT-31"
4  year = 2009
5
6  def print_card():
7      print(f"{name} {surname}", end=" ")
8      print(group)
9
10 print_card()
11
12 def print_card_args(name, surname, group, year):
13     print(f"Name: {name} {surname}\nGroup: {group}\nBirth year: {year}")
14
15 print("--- no parameters, call 1 ---")
16 print_card_args(name, surname, group, year)
17
18 print("--- no parameters, call 2 ---")
19 print_card_args(name, surname, group, year)
20
21 print("--- no parameters, call 3 ---")
22 print_card_args(name, surname, group, year)
23
24
25
26 def print_card_args(name, surname, group, year):
27     print(f"{name} {surname}, {group}, {year}")
28
29 print("--- positional arguments ---")
30 print_card_args("Nikita", "Krilovets", "IT-31", 2009)
31
32
33 print("--- keyword arguments ---")
34 print_card_args(name="Nikita", surname="Krilovets", group="IT-31", year=2009)
35
36
37 print("--- default group ---")
38 print_card_args("Nikita", "Krilovets", "IT-31", 2009)
```

Результат коду:

```
Name: Nikita Krilovets
Group: IT-31
Birth year: 2009
--- no parameters, call 3 ---
Name: Nikita Krilovets
Group: IT-31
Birth year: 2009
--- positional arguments ---
Nikita Krilovets, IT-31, 2009
--- keyword arguments ---
Nikita Krilovets, IT-31, 2009
--- default group ---
Nikita Krilovets, IT-31, 2009
```

Завдання 2. Функції зі значенням, що повертається

Створено функцію `print_age()`, яка обчислює вік за роком народження, але нічого не повертає (`result` дорівнює `None`). Створено функцію `get_age()` зі значенням параметра `current_year` за замовчуванням (2026), яка повертає -1 для некоректного року та вік у протилежному випадку. На основі отриманого віку обчислено вік у місяцях і тижнях, вік станом на 2030 рік та перевірено обробку некоректного року народження (3000).

```
task2.py
1 name = "Nikita"
2 surname = "Krillovets"
3 group = "IT-31"
4 y = 2009
5
6 print(f"{name} {surname}, {group}")
7
8
9 def print_age(year):
10     age = 2026 - year
11     print(f"Age: {age}")
12
13 result = print_age(y)
14
15
16 def get_age(year, current_year = 2026):
17     if year > current_year or year < 0:
18         return -1
19
20     age = current_year - year
21     return age
22     print("after return")
23
24 print(f"print_age returned: {result}")
25
26
27 age = get_age(y)
28 print(f"Age from get_age: {age}")
29
30 months = age * 12
31 weeks = age * 52
32
33 print(f"Age in months: {months}")
34 print(f"Age in weeks: {weeks}")
35
36
37 age_2030 = get_age(y, 2030)
38 print(f"Age in 2030: {age_2030}")
39
40
41 invalid_age = get_age(3000)
42 print(f"Invalid year 3000 gives: {invalid_age}")
43
```

Результат коду:

```
Nikita Krillovets, IT-31
Age: 17
print_age returned: None
Age from get_age: 17
Age in months: 204
Age in weeks: 884
Age in 2030: 21
Invalid year 3000 gives: -1
```

Завдання 3. Функції з анотаціями типів та docstring

Створено декілька функцій з анотаціями типів параметрів і результату: `get_initials()` — формує ініціали, `count_letters()` — рахує входження літери в тексті (з параметром `letter` зі значенням за замовчуванням "a"), `count_vowels()` — рахує голосні, `reverse_text()` — перевертає рядок. Для прізвища виведено ініціали, кількість голосних і приголосних, кількість кожної голосної літери, підрахунок за замовчуванням та перевернутий рядок. Також виведено docstring та анотації типів функції `count_letters` через `__doc__` і `__annotations__`.

```
task3.py
1  name = "Nikita"
2  surname = "Krilovets"
3  c = 9
4
5
6  def get_initials(name: str, surname: str) -> str:
7      return name[0] + "." + surname[0] + "."
8
9
10 def count_letters(text: str, letter: str = "a") -> int:
11     """Return how many times letter occurs in text."""
12
13     count = 0
14
15     for char in text.lower():
16         if char == letter.lower():
17             count += 1
18
19     return count
20
21
22 def count_vowels(text: str) -> int:
23     vowels = 0
24
25     for char in text.lower():
26         if char in "aeiouy":
27             vowels += 1
28
29     return vowels
30
31
32 def reverse_text(text: str) -> str:
33     result = ""
34
35     for char in text:
36         result = char + result
37
38     return result
39
```

```
40
41 print(f"{name} {surname}")
42 print(f"Initials: {get_initials(name, surname)}")
43 print(f"Letters in surname: {c}")
44
45 vowels = count_vowels(surname)
46 consonants = c - vowels
47
48 print(f"Vowels: {vowels}, consonants: {consonants}")
49
50 for vowel in "aeiou":
51     amount = count_letters(surname, letter=vowel)
52     print(f"{vowel}: {amount}")
53
54 print(f"Default letter 'a': {count_letters(surname)}")
55 print(f"Reversed surname: {reverse_text(surname)}")
56
57 print(f"Docstring: {count_letters.__doc__}")
58 print(f"Annotations: {count_letters.__annotations__}")
```

Результат коду:

```
Nikita Krilovets
Initials: N.K.
Letters in surname: 9
Vowels: 3, consonants: 6
a: 0
e: 1
i: 1
o: 1
u: 0
Default letter 'a': 0
Reversed surname: stevolirK
Docstring: Return how many times letter occurs in text.
Annotations: {'text': <class 'str'>, 'letter': <class 'str'>, 'return': <class 'int'>}
```

Завдання 4. Програма з кількома функціями (журнал оцінок)

Побудовано програму з кількох функцій: `read_grade()` зчитує й перевіряє коректність оцінки (цикл із повторним запитом при помилці), `to_letter()` переводить числову оцінку в буквену, `average()` обчислює середнє арифметичне, `count_above()` підраховує оцінки вище заданого рівня, а `print_report()` формує звіт. У функції `main()` для $n = 6$ оцінок запитано значення, після чого викликано `print_report()` для формування підсумкового звіту із середнім балом, найкращою й найгіршою оцінкою та кількістю оцінок вище середнього.

```
task4.py
1 def read_grade(prompt):
2     """Reads a grade from 0 to 100."""
3     while True:
4         grade = input(prompt)
5
6         if not grade.isdigit():
7             print("Error: digits only")
8             continue
9
10        grade = int(grade)
11
12        if grade < 0 or grade > 100:
13            print("Error: the value must be between 0 and 100")
14            continue
15
16        return grade
17
18
19 def to_letter(grade):
20     """Converts a numeric grade to a letter grade."""
21     if grade >= 90:
22         return "A"
23     if grade >= 82:
24         return "B"
25     if grade >= 74:
26         return "C"
27     if grade >= 64:
28         return "D"
29     if grade >= 60:
30         return "E"
31     return "F"
32
33
34 def average(grades):
35     """Returns the arithmetic mean of grades."""
36     return sum(grades) / len(grades)
37
```

```
38
39 def count_above(grades, limit):
40     """Counts grades greater than the given limit."""
41     count = 0
42
43     for grade in grades:
44         if grade > limit:
45             count += 1
46
47     return count
48
49
50 def print_report(name, group, grades):
51     """Prints the student grade report."""
52     avg = average(grades)
53
54     print("--- Report ---")
55     print(f"Student: {name}, group {group}")
56     print("Grades:", grades)
57     print(f"Average: {avg:.2f} -> {to_letter(avg)}")
58     print(f"Best: {max(grades)}, worst: {min(grades)}")
59     print(f"Above average: {count_above(grades, avg)}")
60
61
62 def main():
63     """Runs the main program."""
64     name = "Nikita"
65     group = "IT-31"
66     n = 6
67
68     grades = []
69
70     print(f"{name} {group}")
71
72     for i in range(1, n + 1):
73         grade = read_grade(f"Grade {i} (0-100): ")
74         grades.append(grade)
75
76     print_report(name, group, grades)
77
78     main()
```

Результат коду:

```
Nikita IT-31
Grade 1 (0-100): 70
Grade 2 (0-100): 44
Grade 3 (0-100): 68
Grade 4 (0-100): 55
Grade 5 (0-100): 86
Grade 6 (0-100): 76
--- Report ---
Student: Nikita, group IT-31
Grades: 70 44 68 55 86 76
Average: 66.50 -> D
Best: 86, worst: 44
Above average: 4
```